

PHASE 1 REPORT

Interactive Compiler Interface

A Two-Way Developer-Compiler Interaction Model

Course: BCSE307P Compiler Design Lab

Submitted By: Prashidika Upadhyay (24BCB0167)

Faculty: Dr.Ranjithkumar S

Project Title

Interactive Compiler Interface: A Two-Way Developer–Compiler Interaction Model

The tile conveys the key point of the project that is, modifying the compiler-communicating way in that the coder can have a continuous dialogue with the compiler about the state generated when the code is being compiled.

Abstract

The traditional compiler interaction usually takes the form of source-code submission followed by compilation, diagnostic generation, and then developer interpretation. So, when the compiler comes across the error, the developer typically reads the diagnostic, guesses what it means, decides what kind of change to be made, modifies source code and repeats the compilation process. This work presents a compiler interface with experimental ideas where a diagnostic is considered not to be the end of the interaction but instead a compiler maintains structured information about its analysis state while allowing a programmer to query it, check for associated context, get the list of available alternatives, and implement a limited change.

The system includes the use of a small, controlled programming language and a compiler-style pipeline consisting of these stages: lexical analysis followed by syntax analysis, the abstract syntax tree creation, semantic analysis, symbol-table handling, intermediate representation, and execution. A diagnostic model based on structured data is used to connect these stages to the interaction layer. After an error the developer can run commands like 'explain', 'why', 'show symbols', 'suggest fixes', 'apply 1' directly on the state of the compiler to gain further understanding and possible solutions. When a deterministic and safe repair is available, the user can pick it and then get recompilation done automatically.

In this project, we experiment by doing "what if" kind of controlled implementation, not attempting to be a substitute for a production compiler or a full-feature Integrated Development Environment (IDE) or even an AI coding assistant. The small language and compiler will be a kind of playground where we can implement and directly observe different aspects the interaction model. This way, first phase will be problem definition motivation objectives scope compiler ideas methods design, tools used, and the initial prototype.

1. Problem Statement

1.1 Existing Interaction Pattern

Typically, a compiler undergoes various phases of analysis before either accepting or rejecting a programming language source. At several of these phases, a compiler knows for example, the type of the token it just read, where the syntax error occurred, what the inferred type of the expression is, or even the information from the symbol table available within the current scope. The programmer however is quite limited in her ability to access these features of a compiler directly and because of this mainly relies on the diagnostic messages produced by the compiler.

The typical collaboration process here is a kind of back-and-forth, but at the heart of all this is just a one-line message from the compiler or diagnostic - it's the program which has to provide the next action after each message.

1.2 Problem Identified

This project tries to tackle the issue of limited interaction models around a compiler diagnostic. After the initial error is printed out, the developer may wish to continue discussing that particular problem with the compiler and the diagnostic should not be seen as the final or stopping point.

The project investigates the following question:

What if the compiler's current analysis context could remain addressable for an ongoing interaction with the developer?

1.3 Proposed Problem Resolution

We propose a system which adds a layer capable of interactions around a compiler pipeline that is under control. It holds the structured information of the present diagnostic and the compiler state that may be a factor. The system, while having that information at hand, also translates developers' orders, executes the actions that are explicitly permitted and finally, carries out the steps to recreate the program.

2. Motivation

2.1 Why the Idea

We can be driven by the disparity between the information a compiler keeps to itself, and the way the developer normally interacts with that information. Compiler stages generate a well-organized knowledge of the source code but the developer mostly gets only the final diagnostic message and has to figure out by himself what it is.

2.2 Interaction as an Exchange

This project is based on the hypothesis that error messages can open door to a conversation. Let's say the developer gets the message 'expected;', she may ask the system 'explain' for an explanation. Then the developer might ask 'suggest fixes' and choose from one of the supported options. The system will be able to answer because the needed diagnostic and compiler data are still in memory.

2.3 Design Philosophy

This project isn't trying to alter the way a production compiler is behaving already. On the contrary, this project makes use of a relatively compact smaller compiler as the playground in which experiments can safely be carried out. This setup guarantees that the code will still run efficiently, but also means that the proposed interaction style really be realized not merely shown through static messages.

It is along those lines that, the project, is posing a very specific what-if question: suppose that the compiler were deliberately constructed in such a way that its current state would remain conversable - how then would compiler interaction proceed?

3. Objectives

- Design a tiny programming language with precise lexical and syntactic specification.
- Develop a compiler-style pipeline for lexical analysis, syntax analysis, AST construction, semantic analysis, symbol-table management, intermediate representation, and execution.
- Supplement a detailed compiler diagnostic as structured diagnostic information with phase location diagnostic kind, and contextual compiler environment.
- Create an interface through which a can query a currently active compiler state.
- Use the current diagnostic and compilation for short-sessions so that follow-up commands are related to the same context in the compiler's state.
- Offer deterministic explanation as well as alternatives specific to the situation obtained from compiler information.
- Implement safe and pre-defined edits supported and recompilation and verification.
- Direct an interactive developer, compiler interaction mode and keep it free from turning the project into a general-purpose IDE or AI coding assistant.

4. Scope

4.1 Functional Scope

- An imperative language that is a small one and allows for basic declarations assignments expressions outputs conditional statements, and iterative computations.
- Lexical, syntax, and semantic processing.
- Construction of the abstract syntax tree and symbol tables.
- Generation and examination of the intermediate representation and the possibility of stopping the program execution.
- Reporting, with the help of structured output, and execution of a series of commands interactively.
- Preservation of the context even across sessions.
- A set of possible reasons as to why a program didn't perform as expected and a limited range of operations that may resolve the issue.
- A program is recompiled after making a repair to the cause of an error.

4.2 Language Scope

The language has fewer features compared to general-purpose programming language. It covers this: int float bool, string values; basic arithmetic, relational, and logic operators; variables declaration and assignments; print statements; if/else blocks; and while loops. Full lexical and grammar specifications will be provided in the approach section.

4.3 Interaction Scope

Interaction words have been deliberately kept to a minimum here. The primary commands are giving an explanation, asking the cause or why, looking at a symbol, giving a suggestion, and clicking on a displayed solution. These instructions are all based on the current compiler state and are not free-form natural-language programming assistants.

4.4 Boundaries and Constraints

- The project is just a playground or sandbox compiler, not a full production compiler or a replacement to compiler toolchains.
- The system is not a complete Integrated Development Environment (IDE) or language server either.
- Modifying the source code without any restrictions and on a large scale is forbidden.
- Generation by large language models (LLMs) and repair by autonomous AI are not central topics of the design.
- Only those fixes that have been carefully described in detail by humans, e.g. through deterministic rules, are used.
- The first implementation is focused on one controlled programming language with a command-line interface.

5. Background Study

5.1 Compiler Architecture

Compiler design breaks down the whole process of processing source into separate phases; as a result, a particular phase does one kind of transformation or verification, e.g. lexical analysis converts characters into tokens, syntax analysis verifies grammatical correctness, semantic analysis identifies rules that check meanings, and further phases may build up a new code representation or carry out target execution or target code generation.

5.2 Compiler Diagnostics

Diagnostics help users identify errors that happened during compilation. Useful diagnostics can include source location, problem nature, as well as information of expected or found things. Under the newly proposed system, these diagnostics are treated as data structure which allows future commands to interact with them.

5.3 Existing Developer–Compiler Interaction

Modern development environments can display diagnostics, provide navigation, and in some cases offer quick fixes. This project does not assume that compiler tooling lacks these capabilities. Its focus is instead on an alternative interaction model in which the current compiler analysis context becomes an addressable conversational state during a session.

5.4 Conceptual Gap Explored

The project, as a result, looks at representational and interaction gaps rather than asserting that a compiler algorithm is missing. The compiler still does usual analysis. But the project's main contribution is revealing the compiler's active state and allowing developer interaction with it.

5.5 Sources Consulted

- Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D., *Compilers: Principles, Techniques, and Tools*, 2nd ed., Pearson.
- Appel, A. W., *Modern Compiler Implementation in Java*, 2nd ed., Cambridge University Press.
- Grune, D., van Reeuwijk, K., Bal, H. E., Jacobs, C. J. H., and Langendoen, K., *Modern Compiler Design*, 2nd ed., Springer.
- Compiler Design Laboratory Project Instruction Manual, course-provided Phase 1 project guidelines.

6. Compiler Design Concepts Involved

The project is grounded in core Compiler Design concepts. Each compiler component has a direct role in producing or exposing information used by the interaction layer.

Concept	Role in System	How It Supports Interaction
Lexical Analysis	Recognizes keywords, identifiers, literals, operators, delimiters, and lexical errors.	Token stream and lexical diagnostics.
Syntax Analysis	Validates source structure against the language grammar.	AST construction and syntax diagnostics.
Abstract Syntax Tree	Represents program structure independently of surface syntax.	Structured program context.
Semantic Analysis	Checks declarations, types, scope-related rules, and expression compatibility.	Semantic explanations and diagnostics.
Symbol Table	Stores identifier and type information.	Symbol inspection and context-aware explanations.
Intermediate Representation	Represents valid statements/expressions in a simple three-address-style form.	Inspectable intermediate compiler state.
Execution	Evaluates valid programs in the controlled runtime.	Program result and successful recompilation.
Error Detection	Identifies and classifies compiler failures.	Structured diagnostic object.
Error Recovery / Repair	Supports explicitly defined deterministic corrections.	Candidate repair and apply action.

7. Proposed Methodology

7.1 Overall Processing Method

The system has been built in two modules that are connected together: a compiler pipeline module and an interaction layer module. Compiler module will process the source code normally and at the same time it will expose the internal compiler structures. The interaction layer module will be the one that receives the developer commands and will resolve them about the state of the current session.

1. Get source code and start a compilation session.
2. Do lexical analysis and produce tokens.
3. Carry out syntax check and make AST structure.
4. Conduct semantic check using the symbol table.
5. Generate a simple intermediate representation for inspection.
6. Run valid programs.
7. If there is an error, generate and display a detailed error message.
8. Save the message together with the related compiler parts in the session.
9. Get the developer input and see how it can be done in the current state.
10. Give details about, an inspection, a decision or the result after a developer repair.
11. After a developer has done a fix or any change to a program source, the source is updated and compilation is carried out again.

7.2 Requirement-to-Module Mapping

Design Need	Relevant Syllabus Module
Recognize source elements	Lexer
Validate language structure	Parser
Represent program structure	AST
Check meaning and types	Semantic Analyzer
Track identifiers and types	Symbol Table
Expose intermediate processing	IR Generator
Execute valid programs	Interpreter
Represent compiler problems	Diagnostic Manager
Interpret developer commands	Interaction Manager
Preserve current compiler context	Session Manager
Provide bounded corrections	Repair Engine

7.3 Language Specification

7.3.1 Data Types

Type	Purpose	Example
int	Integer values	10
float	Floating-point values	3.14
bool	Boolean values	true
string	Text values	"hello"

7.3.2 Keywords

int, float, bool, string, print, if, else, while, true, false

7.3.3 Operators and Delimiters

Category	Symbols
Arithmetic	+ - * / %
Relational	< <= > >= == !=
Logical	&& !
Assignment	=
Delimiters	() { } ; ,

7.3.4 Identifiers and Literals

- Identifiers begin with a letter or underscore and may contain letters, digits, and underscores.
- Integer literals consist of decimal digits.
- Floating-point literals contain a decimal point and digits.
- String literals are enclosed in double quotation marks.
- Boolean literals are true and false.

7.3.5 Token Categories

Token Category	Representative Tokens
KEYWORD	int, float, bool, string, print, if, else, while
IDENTIFIER	x, total, message
INT_LITERAL	10, 42
FLOAT_LITERAL	3.14, 0.5
STRING_LITERAL	"hello"
BOOL_LITERAL	true, false
OPERATOR	+, -, *, /, =, ==, <, &&
DELIMITER	(,), {, }, ;

7.3.6 Grammar Rules

```
program      ::= { statement } ;

statement    ::= declaration
               | assignment
               | print_stmt
               | if_stmt
               | while_stmt ;

declaration  ::= type IDENTIFIER "=" expression ";" ;
assignment   ::= IDENTIFIER "=" expression ";" ;
print_stmt   ::= "print" "(" expression ")" ";" ;

if_stmt      ::= "if" "(" expression ")" block
               [ "else" block ] ;

while_stmt   ::= "while" "(" expression ")" block ;
block        ::= "{" { statement } "}" ;

type         ::= "int" | "float" | "bool" | "string" ;

expression   ::= logical_or ;
logical_or   ::= logical_and { "||" logical_and } ;
logical_and  ::= equality { "&&" equality } ;
equality     ::= relational { ("==" | "!=") relational } ;
relational   ::= additive { "<" | "<=" | ">" | ">=" } additive ;
additive     ::= multiplicative { "+" | "-" } multiplicative ;
multiplicative ::= unary { "*" | "/" | "%" } unary ;
unary        ::= ("!" | "-") unary | primary ;
primary      ::= IDENTIFIER
               | literal
               | "(" expression ")" ;

literal      ::= INT_LITERAL
               | FLOAT_LITERAL
               | STRING_LITERAL
               | BOOL_LITERAL ;
```

7.4 Diagnostic and Session Model

A problem related to a compiler is kept as a diagnostic that is in a form suitable for further processing rather than just a string that is printed. Session manager yet preserves and retains not only source but also active diagnostic, the part of compiler code related to the diagnosis and all possible fixes of the error for the ongoing dialogue.

```
Diagnostic {  
  phase  
  type  
  line  
  column  
  symbol  
  expected  
  actual  
  reason  
  actions[]  
}
```

7.5 Interaction and Repair Method

The range of the interaction vocabulary is deliberately limited. The use of one or the other of the commands depends on the current diagnostic or compiler condition. The explanations are made that they only produce the same output every time and the sources of structured data are referenced. Fixes are expressed as clearly outlined actions which contain a short explanation, a location in the source text, and the new text. Any transformation that is not recognized by the system will not be allowed to be used.

```
COMPILER: Error at line 1: expected ';'

YOU: explain
COMPILER: The declaration is missing the ';'
         required by the language grammar.

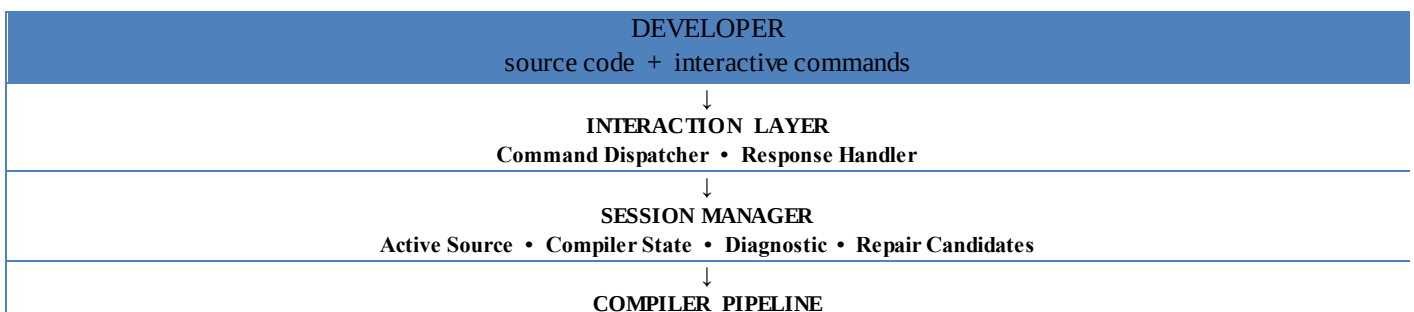
YOU: suggest fixes
COMPILER: [1] Insert ';' at the end of line 1.

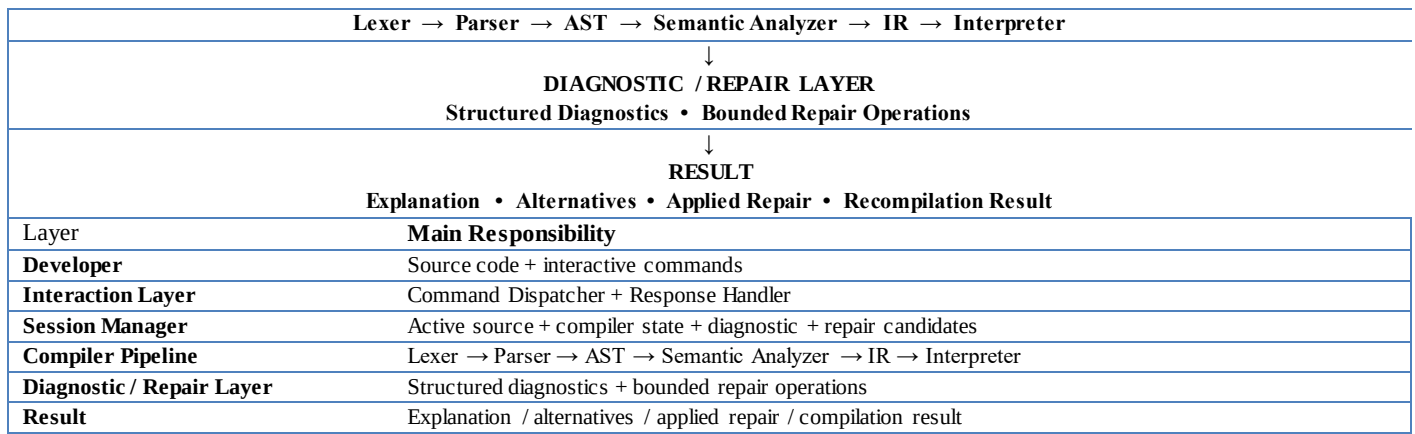
YOU: apply 1
COMPILER: Fix applied. Recompiling...
COMPILER: Compilation successful.
```

8. System Architecture

8.1 High-Level Architecture

The architecture places the interaction layer above a controlled compiler pipeline. The developer communicates with the interaction layer, while the interaction layer accesses structured state produced by the compiler stages.





8.2 Module Responsibilities

Module	Responsibility
Lexer	Converts source characters into classified tokens.
Parser	Consumes tokens according to the grammar and constructs the AST.
AST	Stores the hierarchical program representation.
Semantic Analyzer	Checks declarations, types, and semantic rules.
Symbol Table	Tracks identifiers, types, and scope-related information.
IR Generator	Produces a simple intermediate representation for inspection.
Interpreter	Executes valid programs.
Diagnostic Manager	Creates structured compiler diagnostics.
Interaction Manager	Maps developer commands to compiler-state queries/actions.
Session Manager	Preserves the active compilation context.
Repair Engine	Generates and applies supported deterministic repairs.

8.3 Error Interaction Flow

1. The developer submits source code.
2. A compiler stage detects an error.
3. The diagnostic manager converts the failure into structured state.
4. The session manager stores the active diagnostic and relevant context.
5. The developer asks a command about the problem.
6. The interaction manager resolves the command using the stored state.
7. The system returns an explanation, inspection result, alternatives, or repair action.
8. If a repair is selected, the source is updated and the compiler is invoked again.
9. The new compilation result becomes the current compiler state.

8.4 Design Relationship

The architecture is designed so that the normal compiler pipeline remains untouched in fact. The interaction layer in the new approach doesn't get involved in the work of lexical analysis parsing semantic analysis, and the generation of code (execution). Instead of this functions, the new layer offers a representation of the state that has already been derived from the previous steps of compilation.

This separation enables the compiler and the interaction code layers to be understood individually without losing the benefit of their joint operation as a single system. That distinction is something important of the experiment: the mechanism of interaction is added to compiler components instead of changing compiler concepts.

9. Technology Stack

Component	Technology	Purpose
Programming Language	Python 3.x	Compiler pipeline, interaction layer, and session management
Lexer / Parser	Hand-written Python	Direct control over tokens, grammar, and diagnostics
AST	Python classes / data structures	Structured representation of source programs
Semantic Analysis	Python	Type and declaration checking
Intermediate Representation	Custom three-address-style IR	Inspectable compiler state
Execution	Tree-walking interpreter	Controlled execution of valid programs
Interface	Command-line interface	Developer–compiler interaction
Version Control	Git	Source-code versioning

9.1 Technology Selection Rationale

The reason Python is chosen is that it offers a compact way of writing programs without losing clarity of internal data representations that can be easily looked up while developing and demonstrating the software. An explicitly written lexer and parser grant fine control of the grammar and diagnostic information. For Phase 1 of the project, a simple command-line interface suffices since the focus is on testing the interaction mode and not creating an entire, graphically rich IDE.

10. Initial Prototype

10.1 Prototype Objective

The idea is to make a small-scale prototype first and test the core interaction feature so that we make sure that part is feasible first before completing the entire compiler. Because of that limitation, it employs an intentionally restricted set of language features and goes through a whole mistake - to - interaction - to - recompilation loop.

10.2 Prototype Scenario

We developed the first version which can take in the definition and still recognize the missing semicolon. After analyzing the code, the parser will find a syntax error and a human friendly error report is generated. Then, the communication component of our structure will get the request for clarification, show the possible remedies, let the user choose one and automatically implement it before recompilation.

SOURCE:

```
int x = 10
```

COMPILER:

Error at line 1: expected ';'.

YOU:

explain

COMPILER:

The declaration of 'x' is missing the statement terminator ';' required by the language grammar.

YOU:

suggest fixes

COMPILER:

[1] Insert ';' at the end of line 1.

YOU:

apply 1

COMPILER:

Fix applied. Recompiling...

COMPILER:

Compilation successful.

10.3 Prototype Components

Prototype Component	Demonstrated Function
Lexer	Recognizes the declaration tokens.
Parser	Detects the missing ';'.
Diagnostic Model	Stores the expected token and source location.
Interaction Manager	Accepts explain/suggest/apply commands.
Session Manager	Keeps the active diagnostic available across commands.
Repair Engine	Creates and applies the missing-token insertion.
Recompilation	Verifies the result after the repair.